

All-Clear

프로젝트 메뉴얼

Team : 7

오픈소스기초설계

2025.12.16

[목차]

- 프로젝트 제목, 팀 번호, 팀 구성원 소개
- 기존 서비스 문제 제기
- 프로젝트 GUI
- 설치 방법
- 사용법
- 기대 효과
- Github URL

[프로젝트 제목]

All-Clear

: 보이는 곳과 보이지 않는 곳까지 사각지대 없는 인명 확인 서비스

[팀 번호]

Team 7

[팀 구성원 소개]



이한빛

Object Detection



정서현

Front-end



홍서진(팀장)

Pose 모델 구축



정연재

WiFi 기반 사람 탐지



권현우

Back-end

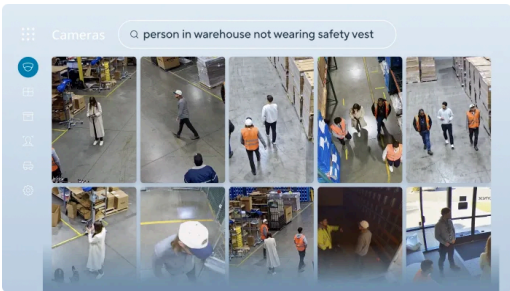
[기존 서비스 문제 제기]

A. CCTV 기반 관제의 한계

- 현재 대부분의 건물은 수십~수백 대의 CCTV를 방제실에서 소수의 관제 요원이 육안으로 모니터링합니다
- NPSA 보고서에 따르면, 동시에 수십~수백 개의 CCTV를 사람이 모니터링 할 경우 주의력 한계로 탐지 성능이 급격히 저하된다고 보고됩니다
- 따라서 사람이 수백 개의 화면을 동시에 감시하기 어렵습니다



- 클라우드 기반 보안시스템 플랫폼 “Verkada”의 경우 AI를 사용하여 산업 현장에서 24시간 실시간으로 모니터링하는 물리 보안 서비스를 제공합니다



- 그러나 화재 발생 시 유독성 연기는 수 초 내 천장부를 가득 채우기 때문에 위와 같은 지능형 카메라를 비롯한 대부분의 CCTV는 연기 확산 이후 영상을 통한 상황 파악 불가합니다
- 결과적으로 관제실은 실내 인원 분포 및 위험도 판단이 불가합니다



[기존 서비스 문제 제기]

B. 정보 부재로 인한 수색 비효율

- 화재 현장 내 인원 존재 여부 및 상태 정보가 사전에 제공되지 않습니다 .
- 따라서 구조대는 모든 공간을 동일 우선순위로 탐색해야 합니다.
- 이로 인해 빈 공간 수색에 시간 소요, 위급 생존자 탐색 지연, 초기 구조 효율 저하가 발생합니다



- 일리노이 대학 연구진들이 개발한 “iRescue”의 경우 이러한 재난 상황에서 무선 네트워크가 존재하는 건물 내부 생존자들의 휴대전화 wifi연결 신호를 추적하여 위치와 상태를 파악하는 시스템을 개발했습니다
- 하지만 휴대전화를 분실/미소지하거나 wifi를 켜두지않은 상태에서는 추적이 불가능합니다

Smartphone app can aid post-disaster rescue

10/4/2012

4 min read

f t in

Picture this: An earthquake strikes, and you're trapped inside a damaged building. You pull out your smart phone to call for help, but you can't get a signal. Suddenly your screen lights up, and an application starts asking questions about your condition. "Are you hurt?" "Are you trapped?" Elsewhere in the building, another victim is unconscious and unable to answer questions, but the app can tell he is lying motionless and transmit that information plus his location to emergency personnel. The information gathered from victims will help first responders organize a quick, efficient rescue operation.



Hyungchul Yoon, left, and Reza Shiftehfar display screen shots from the smart phone app they developed to assist emergency personnel in rescuing people trapped in buildings after disasters.

That's the vision of the University of Illinois researchers who developed the application, which they named iRescue, short for Illinois Rescue. Led by [civil and environmental engineering](#) PhD student Hyungchul Yoon (MS 2012, CEE), the group includes [computer science](#) PhD student Reza Shiftehfar (MS 2009, CEE 09; MS 2010 CS), CEE professor [Bill Spencer Jr.](#), CS professor [Gul Agha](#) and [Beckman Institute](#) professor [Mark Nelson](#). Recent disasters around the world have illuminated the need for better rescue procedures and technology to support them, Yoon said. Before developing iRescue, the team consulted with professionals at the Illinois Fire Service Institute to find out what information would be most helpful for first responders. Location and condition of victims topped the list.

따라서 저희는 Vision AI 기반 이미지 처리 기술과 wifi CSI 신호를 활용해 생존자 자체를 탐지하는 기술을 통합하여 보이는 곳과 보이지 않는 곳까지 사각지대 없는 인명 확인 서비스를 개발하고자합니다.

[AllClear 서비스 소개]

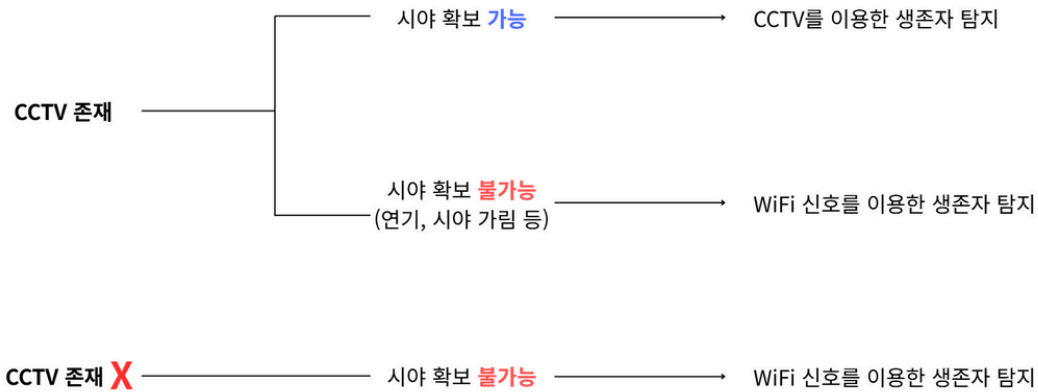
: AllClear는 화재 등 재난 발생 직후 골든 타임 동안 관제 및 구조 판단을 지원하기 위해 설계된 상황 인지 보조 시스템입니다.

서비스 구성 개요

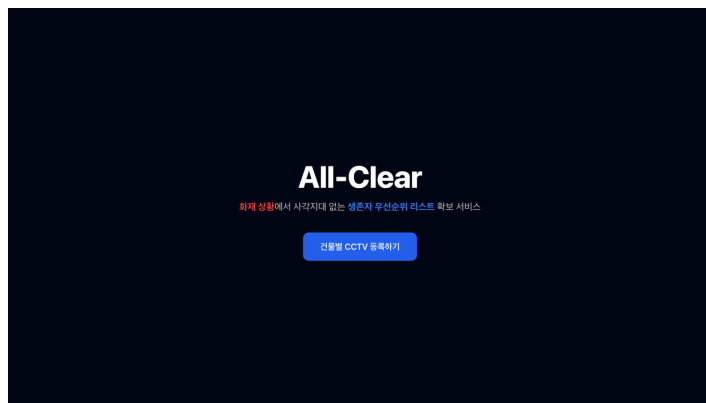
- CCTV 기반 시각 정보
 - Object Detection 모델과 Action Recognition 모델을 활용해 화염과 연기 등 위험 요소를 파악하고 생존자 여부와 상태를 인식
- WiFi CSI 기반 전파 센싱
 - 연기 및 시야 차단 상황에서도 동작
 - 실내 인원 존재 여부 및 움직임 변화 감지에 활용

핵심 기능

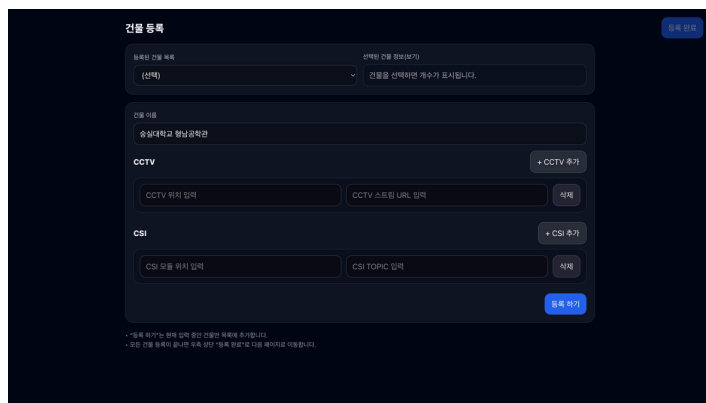
- 생존자 위치 및 위험도 추정
 - 시각 정보와 전파 기반 정보를 통한 통합 분석
 - 공간 단위로 인원 존재 여부 및 상대적 위험도 산출
- 구조 우선순위 지원
 - 분석 결과를 기반으로 구조 대상의 우선순위 제시
 - 구조대가 어디부터 진입해야 하는지에 대한 판단 지원



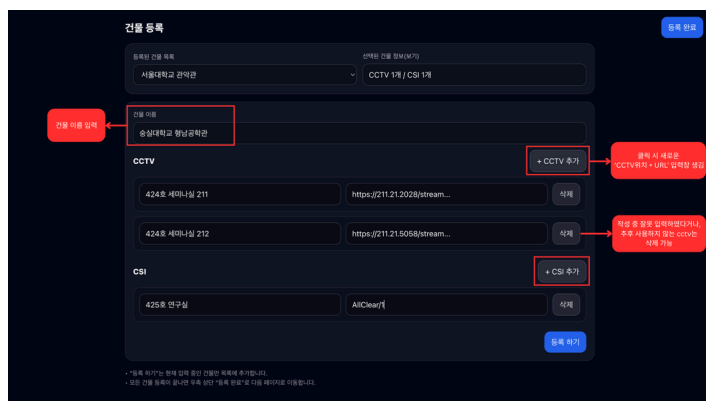
[프로젝트 GUI]



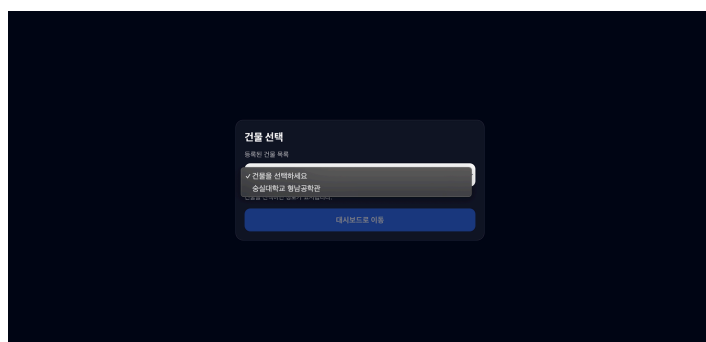
초기 랜딩 페이지에서는 서비스 소개 문구와 함께 건물별 CCTV 등록 시작을 돕습니다.



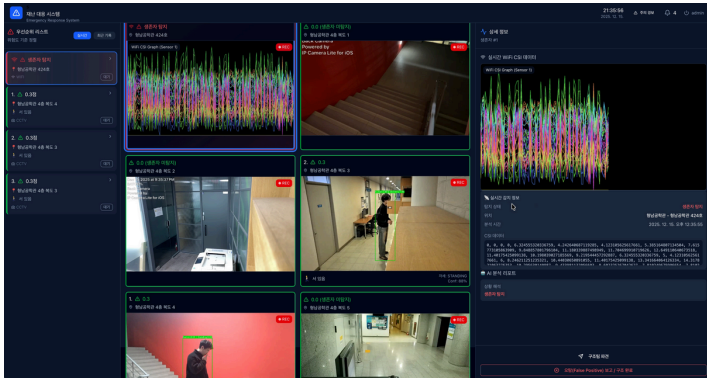
건물 등록 페이지에서는 드롭다운 형태로 건물을 관리하며, 각 건물에 대해 CCTV 위치 및 스트림 URL, CSI 모듈 위치와 Topic을 입력할 수 있습니다.



건물과 건물에 대해 CCTV 위치 및 스트림 URL, CSI 모듈 위치와 Topic을 입력을 완료 후 등록할 수 있습니다. 1개 이상의 건물을 등록하면 등록 완료 버튼을 통해 다음 페이지로 넘어갈 수 있습니다



등록된 건물 목록 중 하나를 선택하여 해당 건물의 대시보드로 이동할 수 있습니다.



선택된 건물의 CCTV 영상과 센서 기반 생존자 우선순위 리스트를 하나의 대시보드에서 통합적으로 확인할 수 있습니다.

[설치 방법 (로컬 환경에서 서버 구동 방법)]

A. Github Repository Clone 및 사전 준비

B. 서버 실행 및 테스트

C. 프론트 연동

A. Github Repository Clone 및 사전 준비

Step 1: 사전 요구사항 체크

1. OS: Window/macOS/Linux
2. Java: JDK 21
3. Build: Gradle Wrapper(8.x)
4. Python: 3.10+ (FastAPI용), uvicorn, fastapi, 모델 의존 패키지
5. DB: 기본 Oracle 설정 (H2 임시 사용 시 별도 local 프로파일 필요)

Step 2: 코드 받기

1. 터미널 또는 Powershell을 열고 원하는 폴더로 이동
2. Repository 클론

\$ git clone https://github.com/All-Clear-SSU/server all-clear-server

```
mailofh@gwonhyeon-uui-MacBookAir Documents % git clone https://github.com/All-Clear-SSU/server all-clear-server
Cloning into 'all-clear-server'...
remote: Enumerating objects: 330, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 330 (delta 2), reused 12 (delta 0), pack-reused 300 (from 1)
Receiving objects: 100% (330/330), 33.60 MiB | 10.92 MiB/s, done.
Resolving deltas: 100% (130/130), done.
```

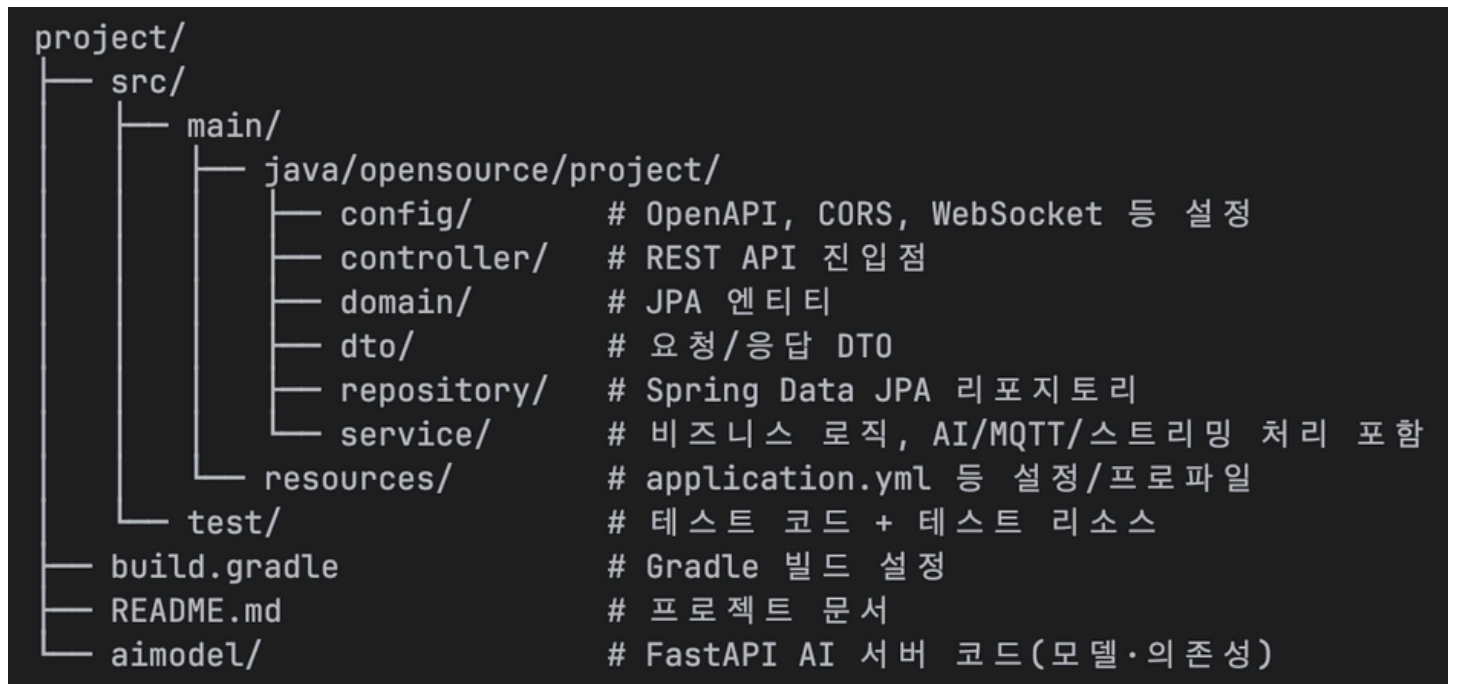
3. 프로젝트 루트 폴더로 이동

\$ cd all-clear-server

```
mailofh@gwonhyeon-uui-MacBookAir Documents % cd all-clear-server

mailofh@gwonhyeon-uui-MacBookAir all-clear-server % ls
README.md      build.gradle   gradlew        settings.gradle
aimodel        gradle        gradlew.bat    src
```

4. Repository 구조 확인



Step 3: 환경 변수 설정 (중요)

Why?

- 기존 서비스: 수정된 코드가 Github Repository에 Push될 때, 클라우드 상에 자동 배포되는 구조
→ 외부 노출에 민감한 변수에 대해서는 다음 사진과 같이 Github Secret을 이용해 환경변수로 주입하는 방식을 사용 중
- 로컬 서버 실행: 필요한 환경변수들을 직접 주입

Repository secrets

New repository secret

Name 	Last updated
 DB_PASSWORD	last month  
 DB_URL	last month  
 DB_USERNAME	last month  
 EC2_HOST	last month  
 EC2_PEM_KEY	last month  
 EC2_PEM_KEY_BASE64	last month  
 MQTT_BROKER_URL	last month  
 MQTT_CLIENT_ID	last month  
 MQTT_ENABLED	2 weeks ago  
 MQTT_TOPIC	4 days ago  

1. 주입이 필요한 환경 변수 확인

- Spring Boot(백엔드) 환경 변수
- DB_URL**: DB 연결을 위한 엔드포인트(Oracle 예: jdbc:oracle:thin:@127.0.0.1:1521/XE; H2 테스트 예: jdbc:h2:mem:testdb)
 - DB_USERNAME**: DB 사용자 이름 (예: admin)
 - DB_PASSWORD**: DB 비밀번호
 - SERVER_BASE_URL**(기본 http://localhost:8080): 백엔드에서 외부용 링크/HLS 등을 만들 때 쓰는 주소이므로, 로컬에서 서버 실행 시, http://localhost:8080으로 두면 됩니다. 즉, 브라우저가 직접 접근할 주소와 일치시키세요.
 - AI_API_BASE_URL**(기본 http://localhost:8000): 백엔드가 FastAPI에 호출을 보낼 때 쓰는 base URL 이므로, FastAPI를 8000에서 돌리기 때문에 마찬가지로 http://localhost:8000으로 두면 됩니다.
 - MQTT_ENABLED**(기본 false): MQTT 브로커 서버와 통신을 시작할지 안할지를 boolean 값으로 결정하는 변수이기 때문에 true로 설정시 아래 3가지 MQTT 관련 변수도 반드시 설정해주어야 합니다.
 - MQTT_BROKER_URL**: MQTT 브로커 접속에 필요한 URL(예: tcp://127.0.0.1:1883)
 - MQTT_CLIENT_ID**: 클라이언트 ID(예: all-clear-local-1)

c. **MQTT_TOPIC**: 구독/발행 토픽(예: all-clear/alert)

- FastAPI(AI) 환경 변수

1. **SPRING_BOOT_URL**(기본 http://localhost:8080): 백엔드가 AI로부터 결과를 받을 때 사용하는 백엔드 주소, 마찬가지로 로컬에서 실행하기 때문에 그대로 두면 됩니다.
2. **HLS_BASE_URL**(기본 http://localhost:8080/streams): HLS 스트림 URL 생성 시 사용하는 베이스 주소, 마찬가지로 주소는 그대로 두되(http://localhost:8080), 어떤 디렉터리에 저장할지는 (/streams) 직접 설정해주면 됩니다.

참고: 모델 경로는 상대경로로 지정되어 있기 때문에 별도 환경변수 주입 필요 없음

2. 환경 변수 설정 영구 적용시키기

Case 1: Mac / Linux

1. 로컬 홈(~/) 경로에서 환경파일 생성

```
cat > ~/.allclear-env <<'EOF'
# Spring Boot(백엔드) 용
export DB_URL="jdbc:oracle:thin:@127.0.0.1:1521/XE"
export DB_USERNAME="scott"
export DB_PASSWORD="tiger"
export SERVER_BASE_URL="http://localhost:8080"
export AI_API_BASE_URL="http://localhost:8000"
export MQTT_ENABLED="true"
export MQTT_BROKER_URL="tcp://127.0.0.1:1883"
export MQTT_CLIENT_ID="all-clear-local-1"
export MQTT_TOPIC="allclear/alert"
# FastAPI(AI)용:
export SPRING_BOOT_URL="http://localhost:8080"
export HLS_BASE_URL="http://localhost:8080/streams"
EOF
```

```
mailofh@gwonhyeon-uui-MacBookAir Documents % cat > ~/.allclear-env <<'EOF'
export DB_URL="jdbc:oracle:thin:@127.0.0.1:1521/XE"
export DB_USERNAME="scott"
export DB_PASSWORD="tiger"
export SERVER_BASE_URL="http://localhost:8080"
export AI_API_BASE_URL="http://localhost:8000"
export MQTT_ENABLED="true"
export MQTT_BROKER_URL="tcp://127.0.0.1:1883"
export MQTT_CLIENT_ID="all-clear-local-1"
export MQTT_TOPIC="allclear/alert"
export SPRING_BOOT_URL="http://localhost:8080"
export HLS_BASE_URL="http://localhost:8080/streams" EOF
```

2. 셸 초기화 파일에 추가 후 적용

```
echo 'source ~/.allclear-env' >> ~/.zshrc # bash면 ~/.bashrc
```

```
source ~/.zshrc
```

```
mailofh@gwonhyeon-uui-MacBookAir Documents % echo 'source ~/.allclear-env' >> ~/.zshrc source ~/.zshrc ]
```

Case 2: Windows

1. PowerShell에서 다음 명령어 수행

```
setx DB_URL "jdbc:oracle:thin:@127.0.0.1:1521/XE"
```

```
setx DB_USERNAME "scott"
```

```
setx DB_PASSWORD "tiger"
```

```
setx SERVER_BASE_URL "http://localhost:8080"
```

```
setx AI_API_BASE_URL "http://localhost:8000"
```

```
setx MQTT_ENABLED "true"
```

```
setx MQTT_BROKER_URL "tcp://127.0.0.1:1883"
```

```
setx MQTT_CLIENT_ID "all-clear-local-1"
```

```
setx MQTT_TOPIC "allclear/alert"
```

```
# FastAPI(AI)용:
```

```
setx SPRING_BOOT_URL "http://localhost:8080"
```

```
setx HLS_BASE_URL "http://localhost:8080/streams"
```

3. 적용 여부 확인

Case 1: Mac / Linux

```
env | grep DB_URL
```

```
env | grep MQTT_TOPIC
```

등으로 확인

```
mailofh@gwonhyeon-uui-MacBookAir server % env | grep DB_URL
DB_URL=jdbc:oracle:thin:@127.0.0.1:1521/XE
```

Case 2: Windows

2. 새로운 PowerShell을 열고 적용 여부 확인

```
echo $env:DB_URL
```

```
echo $env:MQTT_BROKER
```

등으로 확인

- 주의 사항 재정리

- 프론트까지 로컬인 상황이기 때문에 SERVER_BASE_URL, AI_API_BASE_URL 등을 http://localhost:8000 그대로 사용하세요. (다른 기기/포트 접근 시 실제 접근 가능한 주소로 변경하세요)
- DB를 H2로 테스트 시 application-local.yml로 driver/dialect를 H2로 override 후, gradle 혹은 jar로 실행시 --spring.profiles.active=local 옵션을 추가하세요.

```
application-local.yml
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    username: sa
    password: password
    driver-class-name: org.h2.Driver

jpa:
  properties:
    hibernate:
      dialect: org.hibernate.dialect.H2Dialect
  hibernate:
    ddl-auto: update

h2:
  console:
    enabled: true
    path: /h2-console
```

B. 서버 실행 및 테스트

Step 1: 의존성 설치 및 빌드

`./gradlew tasks` // Gradle Wrapper로 이 프로젝트에 정의된 주요 작업 목록을 보여줍니다. Wrapper가 정상 동작하는지, 어떤 태스크가 있는지 확인할 때 사용합니다.

`./gradlew build` // 소스 컴파일 → 테스트 실행 → JAR 생성까지 한번에 수행합니다. 의존성도 자동으로 내려받습니다.

<./gradlew build 성공시 예시>

```
mailofh@gwonhyeon-uyi-MacBookAir core % ./gradlew build
Starting a Gradle Daemon, 1 incompatible and 1 stopped Daemons could not be reused, use --status for details
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

BUILD SUCCESSFUL in 16s
7 actionable tasks: 5 executed, 2 up-to-date
```

Step 2: Spring Boot(백엔드) 서버 실행

1. 서버 실행

`./gradlew bootRun` // Gradle이 애플리케이션을 빌드한 뒤 JVM을 띄워 곧바로 서버를 구동

```
mailofh@gwonhyeon-uyi-MacBookAir core % ./gradlew bootRun

> Task :bootRun

  ____  _  / ____ \  / ____ \  / ____ \  / ____ \  / ____ \  / ____ \
 / ___ \| | | |  _ \| |  _ \| |  _ \| |  _ \| |  _ \| |  _ \| |  _ \|
| |   \| |_| | |_) | | |_) | | |_) | | |_) | | |_) | | |_) | | |_) |
| |___| |  __| | |  __| | |  __| | |  __| | |  __| | |  __| | |  __|
\_____/|_|_|_|_____|_|_|_____|_|_|_____|_|_|_____|_|_|_____|_|_|_____|
:: Spring Boot ::                               (v3.5.6)
```

또는

`./gradlew bootJar` // 실행하지 않고 배포용 JAR 파일을 만듭니다. 결과물은 build/libs/ 디렉터리에 있는 project-0.0.1-SNAPSHOT.jar(설정에 따라 이름 상이할 수 있음)입니다. 실행하려면 별도로 `java -jar build/libs/...jar`를 호출해야 합니다.

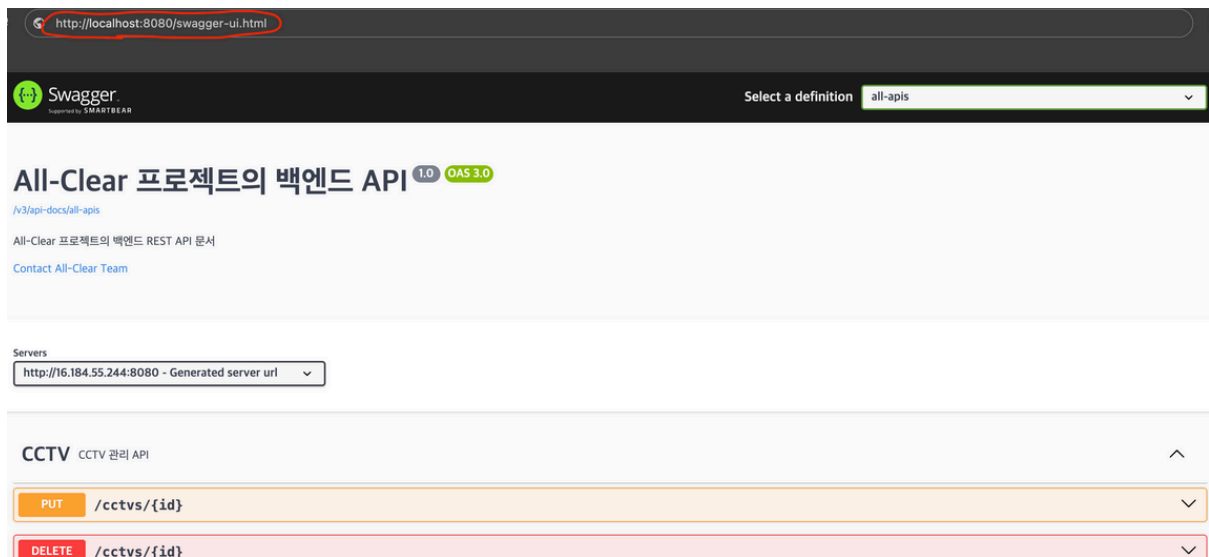
```
mailofh@gwonhyeon-uyi-MacBookAir core % ./gradlew bootJar

BUILD SUCCESSFUL in 1s
4 actionable tasks: 4 up-to-date
```

2. 실행 확인

Swagger: <http://localhost:8080/swagger-ui.html>

해당 Swagger UI 에 접속이 가능하다면 정상적으로 Spring Boot 서버가 실행된 것입니다.



Step 3: FastAPI(AI) 서버 실행

1. 서버 실행

`cd aimodel` // fastapi 서버 실행을 위한 디렉터리 진입

`python -m venv .venv` // 프로젝트별 패키지 격리를 위해 현재 위치에 가상환경 폴더 .venv 생성

`source .venv/bin/activate` # Windows: `.venv\Scripts\activate` // 방금 만든 가상환경을 활성화해서 이후 pip 설치가 이 환경에만 적용되도록 함

`pip install -r requirements.txt` //FastAPI 서버가 필요로 하는 패키지들을 요구사항 파일에 맞춰 설치

`uvicorn main:app --reload --host 0.0.0.0 --port 8000` // main.py의 app 객체를 Uvicorn으로 실행. --reload는 코드 변경 시 자동 재시작, --host 0.0.0.0은 모든 인터페이스에서 수신, --port 8000은 포트 지정 (백엔드와 통신할 때 AI_API_BASE_URL 기본값과 일치)

```
mailofh@gwonhyeon-uui-MacBookAir all-clear-server % cd aimodel
```

```
mailofh@gwonhyeon-uui-MacBookAir aimodel % python -m venv .venv
```

```
mailofh@gwonhyeon-uui-MacBookAir aimodel % source .venv/bin/activate
```

```
mailofh@gwonhyeon-uui-MacBookAir aimodel % pip install -r requirements.txt
```

```

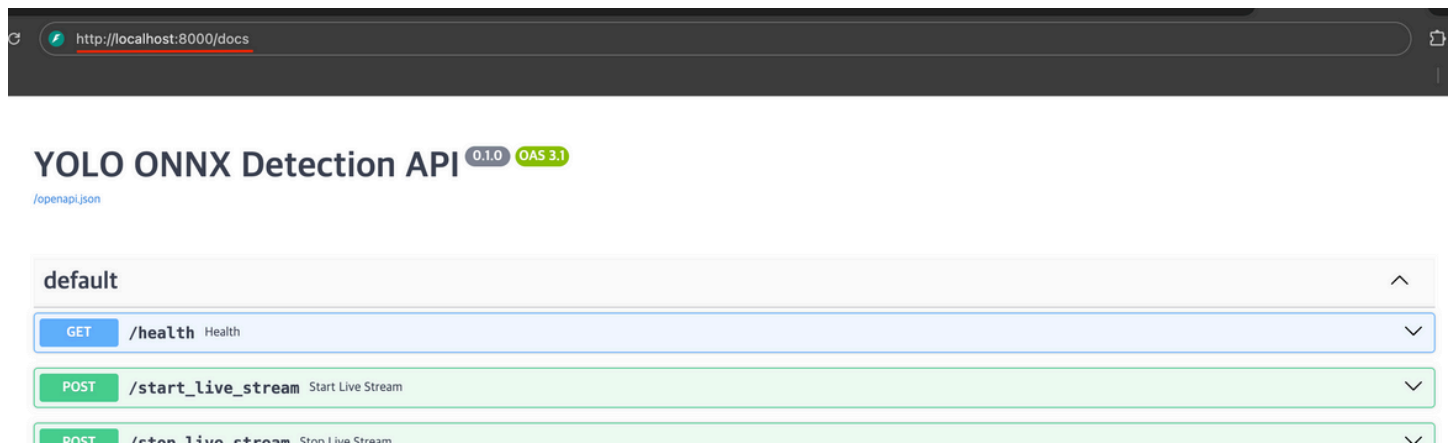
mailofh@gwonhyeon-uu-MacBookAir aimodel % python -m uvicorn main:app
--reload --host 0.0.0.0 --port 8000
INFO: Will watch for changes in these directories: ['/Users/mailofh/
Documents/study/server/aimodel']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [12345] using StatReload
INFO: Started server process [12346]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

2. 실행 확인

Docs: <http://localhost:8000/docs>

해당 Docs 문서 UI에 접속 가능하다면 정상적으로 FastAPI 서버가 실행된 것입니다.



Step 4: Spring Boot <-> FastAPI 간 응답 성공 여부 확인

Spring Boot에서 FastAPI로 응답을 요청하는 API 실행 후 로그 확인 후 다음과 같이 status=200 이 반환되면 정상적으로 백엔드와 AI 모델 서버간 통신이 수행되고 있는 것입니다.

<Spring Boot 로그>

```

2025-12-16T19:55:26.412+09:00 INFO 12345 --- [ctor-http-nio-3]
o.p.s.service.ObjectDetectionApiClient : [AI REQUEST] POST http://
localhost:8000/predict_image cctvId=12
2025-12-16T19:55:26.921+09:00 INFO 12345 --- [ctor-http-nio-
3] o.p.s.service.ObjectDetectionApiClient : [AI RESPONSE]
status=200 body={"detections":[{"label":"person","score":0.92,"bbox":
[123,45,210,330]}]}
2025-12-16T19:55:26.925+09:00 INFO 12345 --- [ctor-http-nio-3]
o.p.s.service.AIDetectionProcessorService: Saved detection: cctvId=12
count=1

```


C. 프론트 연동

Step 1: 사전 요구사항 체크

1. Node.js 20+ (추천)
2. 백엔드 CORS에 프론트 포트(기본 3000)가 허용돼 있어야 함
3. 백엔드 서버는 로컬에서 실행 중이어야 함

Step 2: 코드 받기

1. 터미널 또는 Powershell을 열고 원하는 폴더로 이동
2. Repository 클론
\$ git clone https://github.com/All-Clear-SSU/client all-clear-client

```
mailofh@gwonhyeon-uui-MacBookAir Documents % git clone https://github.com/All-Clear-SSU/client all-clear-client
Cloning into 'all-clear-client'...
remote: Enumerating objects: 245, done.
remote: Counting objects: 100% (245/245), done.
remote: Compressing objects: 100% (188/188), done.
remote: Total 245 (delta 108), reused 177 (delta 51), pack-reused 0 (from 0)
Receiving objects: 100% (245/245), 194.53 KiB | 12.97 MiB/s, done.
Resolving deltas: 100% (108/108), done.
```

3. 프로젝트 루트 폴더로 이동
\$ cd all-clear-server

```
mailofh@gwonhyeon-uui-MacBookAir Documents % cd all-clear-client
mailofh@gwonhyeon-uui-MacBookAir all-clear-client % ls
README.md      netlify.toml      postcss.config.js  tailwind.config.js  tsconfig.node.json
eslint.config.js  package-lock.json  public             tsconfig.app.json   vite.config.ts
index.html      package.json      src                tsconfig.json
```

4. Repository 구조 확인


```

client/
├── src/                # React/Vite 소스
│   ├── assets/        # 정적 자원
│   ├── components/    # UI 컴포넌트
│   ├── lib/           # api.ts, socket.ts 등 공용 로직
│   ├── pages/         # 라우트 페이지
│   └── styles/        # 전역 스타일(있다면)
├── public/            # 공개 정적 파일
├── package.json        # 스크립트/의존성
├── vite.config.ts      # Vite 설정
├── tsconfig*.json     # TS 설정
├── tailwind.config.js  # Tailwind 설정
├── netlify.toml        # 배포/프록시 설정(사용시)
└── README.md          # 프론트 문서

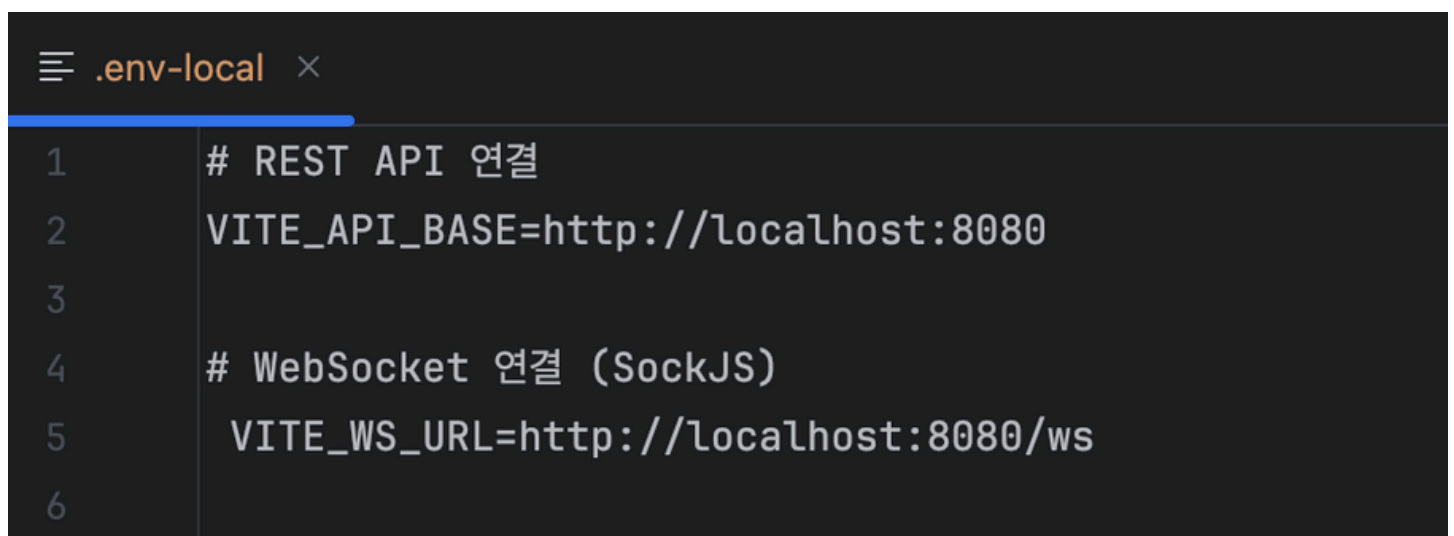
```

Step 3: 환경 변수 설정 및 의존성 설치

1. .env.local 파일 생성하고, 다음을 입력하여 환경변수 주입합니다

```
VITE_API_BASE=http://localhost:8080 # REST 호출
```

```
VITE_WS_URL=http://localhost:8080/ws # STOMP/SockJS WebSocket
```



The screenshot shows a code editor with a tab labeled `.env-local`. The file content is as follows:

```

1  # REST API 연결
2  VITE_API_BASE=http://localhost:8080
3
4  # WebSocket 연결 (SockJS)
5  VITE_WS_URL=http://localhost:8080/ws
6

```

2. npm 설치

```
npm install # npm 설치
```

```
mailofh@gwonhyeon-uu1-MacBookAir all-clear-client % npm install
npm warn deprecated yaeti@0.0.6: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.

added 409 packages, and audited 410 packages in 5s

68 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Step 4: 프론트 서버 실행

```
npm run dev -- --host --port 3000
```

접속 주소: <http://localhost:3000>

백엔드 서버가 켜져 있어야 API와 WebSocket 연결이 성공합니다.

```
mailofh@gwonhyeon-uu1-MacBookAir all-clear-client % npm run dev -- --host --port 3000

> allclear@0.0.0 dev
> vite --host --port 3000

VITE v7.1.11 ready in 798 ms

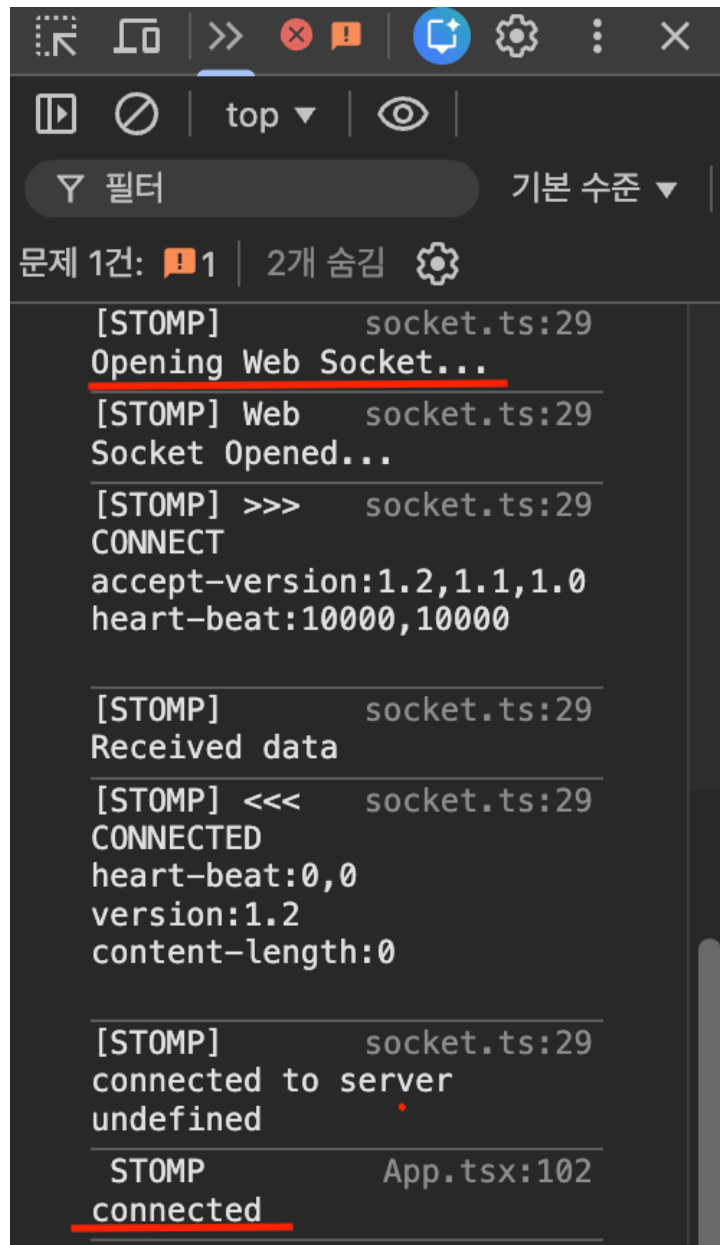
→ Local:   http://localhost:3000/
→ Network: http://192.168.0.41:3000/
→ press h + enter to show help
```

Step 5: 백엔드 서버와 연동 확인

1. WebSocket 연결 확인

프론트 페이지가 열린 상태에서 브라우저의 개발자 도구 → Console 클릭하여 콘솔창을 띄웁니다.

콘솔창에 [STOMP] Opening Web Socket... → [STOMP] connected 메시지가 뜨면 백엔드와 WebSocket 연결이 성공한 것입니다.



2. HLS / 스트림 연결 확인

- curl 을 이용해 확인

```
curl -I http://localhost:8080/streams/cctv1/playlist.m3u8
```

터미널에서 위 명령어를 입력하였을 때 200(Success) 응답이 반환되면 백엔드와 HLS 스트림 연결이 성공한 것입니다.

```
mailofh@gwonhyeon-uui-MacBookAir all-clear-client % curl -I http://localhost:8080/streams/cctv1/playlist.m3u8
```

```
HTTP/1.1 200
Vary: Origin
Vary: Access-Control-Request-Method
Vary: Access-Control-Request-Headers
Cache-Control: no-store
Last-Modified: Tue, 16 Dec 2025 11:58:21 GMT
Accept-Ranges: bytes
Content-Type: application/vnd.apple.mpegurl
Content-Length: 235
Date: Tue, 16 Dec 2025 11:58:21 GMT
```

- 실제 라이브 스트리밍으로 확인

Swagger UI 내 POST /live-stream/start API를 이용해 CCTVID = 1 로 스트리밍을 시작하고,

POST /live-stream/start 라이브 스트리밍 시작

CCTV ID와 위치 ID를 받아 라이브 스트리밍을 시작합니다.

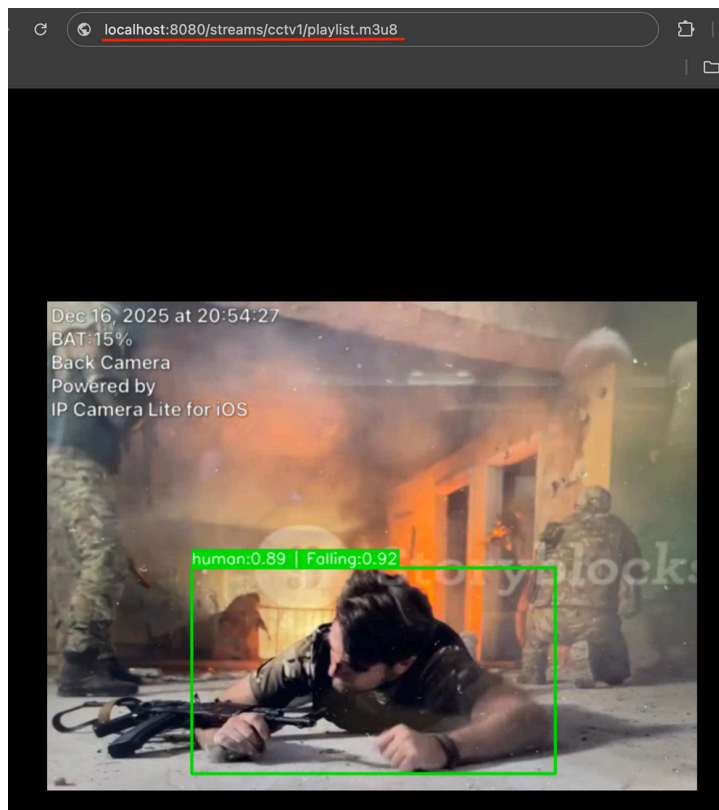
Parameters

Name	Description
cctvid • required integer(\$int64) (query)	CCTV ID
locationId • required integer(\$int64) (query)	위치 ID
confThreshold number(\$double) (query)	객체 탐지 신뢰도 임계값 (기본값: 0.3)
poseConfThreshold number(\$double) (query)	자세 분류 신뢰도 임계값 (기본값: 0.3)

Execute Clear

<http://localhost:8080/streams/cctv1/playlist.m3u8>

브라우저 주소창에 위 주소를 입력하였을 때 스트리밍이 정상적으로 진행되면 백엔드 서버와 HLS 스트림 연결이 정상적으로 수행된 것입니다.



- 문제 발생 가능 상황 정리
 - 콘솔창 + 네트워크창을 통해 오류 메시지 확인 가능
 - API 404/네트워크 에러: .env.local의 VITE_API_BASE가 백엔드 실제 주소와 일치하는지 확인하고, 백엔드 서버 실행 여부를 확인하세요.

- WebSocket 실패: VITE_WS_URL이 올바른지, 백엔드의 /ws 엔드포인트가 열려 있는지 확인하세요.
- CORS 오류: 백엔드 CORS 설정에 http://localhost:3000 추가가 필요합니다.

[하드웨어 세팅 방법]

A. WiFi 모듈 세팅

B. WiFi 학습 및 추론 실행

A. WiFi 모듈 세팅

준비물

- ESP 보드 1대
- USB 케이블
- 공유기
- 노트북/PC

Step 1: ESP-IDF 설치

- Windows
 1. <https://dl.espressif.com/dl/esp-idf/> ► ESP-IDF Windows Installer 다운로드
 2. “Full Install” 선택 → Next만 눌러 마침까지 진행
 3. 설치 완료 후 ESP-IDF Command Prompt 아이콘 실행
- macOS / Linux
 1. `$ mkdir -p ~/esp && cd ~/esp`
 2. `$ git clone --recursive https://github.com/espressif/esp-idf.git`
 3. `$ cd esp-idf`
 4. `$ ./install.sh`
 5. `$ source export.sh`

Step 2: 코드 받기

1. 터미널 또는 Powershell을 열고 원하는 폴더로 이동
2. 레포지토리 클론
`$ git clone https://github.com/All-Clear-SSU/wifi_presence`
3. 프로젝트 루트 폴더로 이동
`$ cd CSI-python`
4. 의존성 설치
`$ pip install -r requirements.txt`

Step 3: WiFi 정보 입력

config.json 파일에 직접 설치할 모듈과 연결될(사용하고 있는) 와이파이 정보를 입력해주세요.

```
{
  "wifi_ssid": "YOUR_WIFI_SSID",
  "wifi_password": "YOUR_PASSWORD",
  "publish_topic": "ALLCLEAR/SSU/BLDG-A/F03/RM-301/ESP01",
  "wifi_mac": "AA:BB:CC:DD:EE:FF"
}
```

- **wifi_ssid**: 연결할 공유기 이름(2.4GHz)
- **wifi_password**: 위 공유기의 비밀번호
- **publish_topic**: 장치가 발행할 토픽
권장 형식: ALLCLEAR/지명/건물번호/층/방
- **wifi_mac**: 와이파이 MAC 주소

Step 4: ESP 보드에 펌웨어 업로드

노트북 혹은 컴퓨터에 ESP32 모듈을 오른쪽 사진과 같이 연결해주세요.

펌웨어를 ESP 보드에 업로드 해주세요.

```
$ idf.py -p flash monitor
```

부팅 로그에 연결된 SSID, 브로커, 퍼블리시 토픽이 출력되면 정상입니다.



Step 4: 연결 확인(선택 사항)

MQTT 브로커에서 토픽을 구독해 데이터 수신을 확인해주세요.

```
$ mosquitto_sub -h allclear.sytes.net -p 4341 -t 'ALLCLEAR/본인이 설정한 토픽' -v
```

B. WiFi 학습 및 추론 실행

Step 1: 원하는 방에 WiFi 모듈 설치

존재 감지를 원하는 방에 ESP 모듈을 아래 사진과 같이 설치해주세요.



Step 2: 베이스라인 잡기 (캘리브레이션)

사람 미존재 상태의 WiFi 신호 특성을 기준값으로 설정하기 위해 실행합니다.

!사람이 없는 환경에서 실행하세요!

```
$ python3 calibrate.py
```


Step 3: 실시간 추론 실행

캘리브레이션된 기준값과 현재 WiFi 신호를 비교하여 사람 존재 여부를 실시간 판단하기 위해 실행합니다.

```
$ python3 realtime_infer.py
```

빠른 점검

Q. “연결이 안돼요”

A. 브로커 host/port 오타/방화벽 확인이 필요합니다.

Q. “예측이 들쭉날쭉해요”

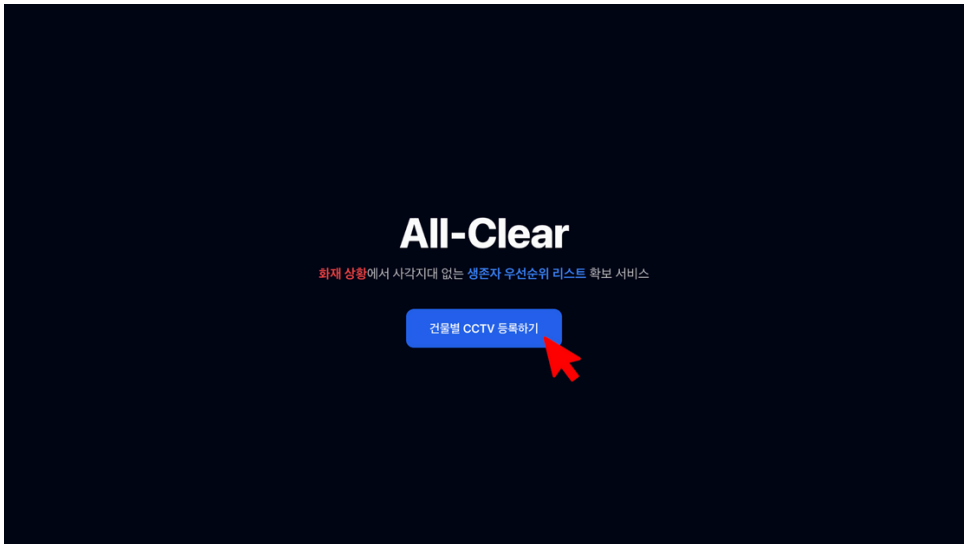
A. 사람이 없는 시간대에 빈 방에서 다시 캘리브레이션을 진행해보세요!

[사용법]

- A. 웹페이지 사용법
- B. 메인 웹페이지 설명
- C. 백엔드 서버 관리 방법

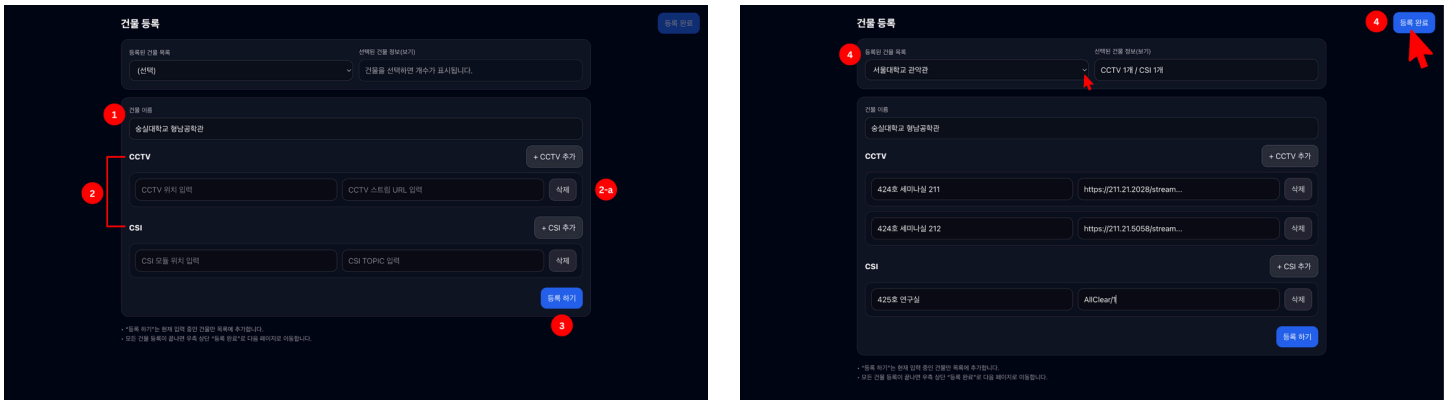
A. 웹페이지 사용법

Step 1: 랜딩 페이지 (All-Clear + 버튼)



사용자는 초기 화면에서 ‘건물별 CCTV 등록하기’ 버튼을 클릭하여 건물 및 센서 등록 단계로 이동합니다.

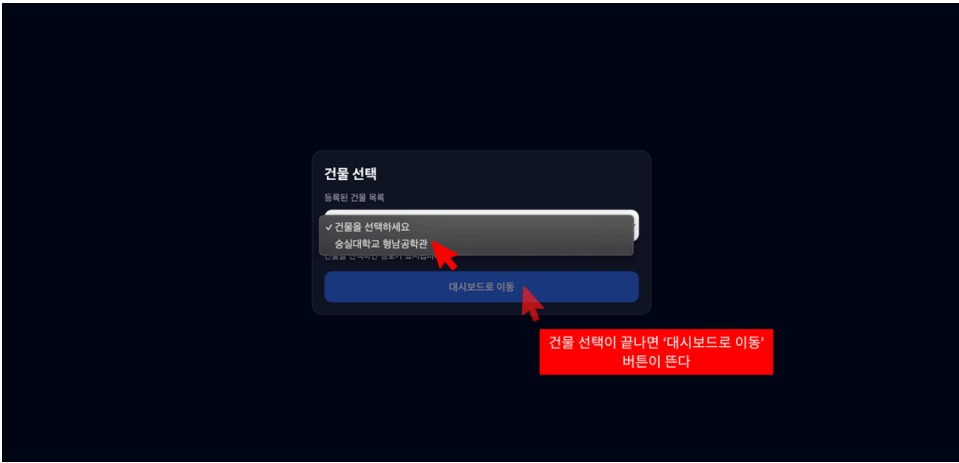
Step 2: 건물 + CCTV + CSI 등록 페이지



1. 사용자는 등록할 건물 이름을 입력합니다.
2. CCTV 또는 CSI 정보를 최소 하나 이상 ‘CCTV 위치 + URL’ 입력합니다.
 - a. 잘못 입력 하였을 시 오른쪽 ‘삭제’ 버튼 클릭 가능합니다.
3. CCTV 또는 CSI 정보 입력이 끝났다면 하단에 뜨는 ‘등록하기’ 버튼을 클릭하여 건물을 등록합니다.
4. 동일한 방식으로 여러 건물을 연속 등록할 수 있습니다. → 등록한 건물과 정보는 좌측 상단의 드롭다운에서 확인이 가능합니다.

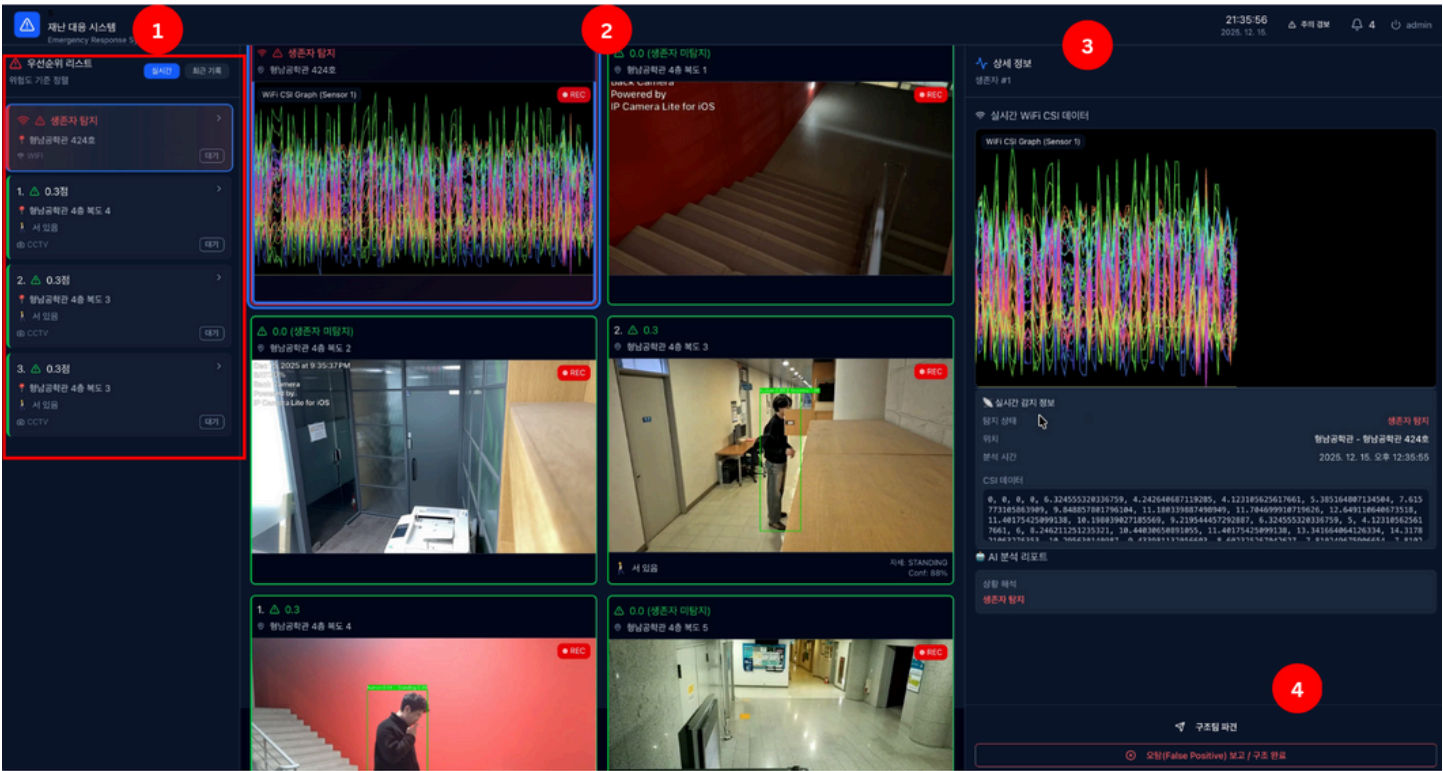
5. 등록하고자 하는 모든 건물을 등록하였다면, 우측 상단에 ‘등록 완료’ 버튼을 클릭할 시 다음 페이지로 넘어갑니다.

Step 3: 건물 선택 드롭다운 페이지



사용자는 드롭다운에서 모니터링할 건물을 선택합니다. 선택한 건물의 대시보드 화면으로 이동하게 됩니다.

Step 4: CCTV 메인 대시보드



대시보드 화면을 통해 실시간 CCTV 영상과 생존자 우선순위 정보를 확인하여 구조 판단에 활용합니다.

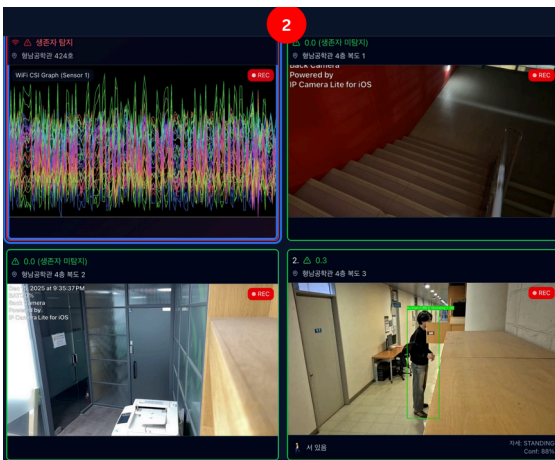
B. 메인 웹페이지 설명

1. 좌측 보드: 구조 우선순위 리스트



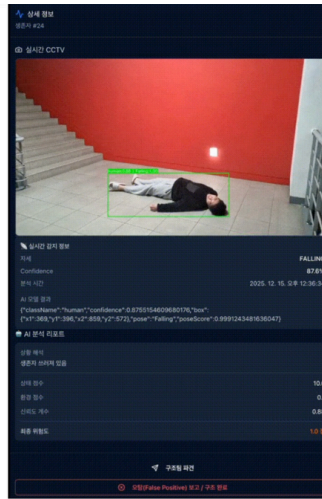
- **실시간 탭**
 - **정렬:** 생존자의 자세, 화염/연기 유무로 계산된 위험도 점수에 따라 내림차순 자동 정렬
 - **인터랙션:** 리스트 클릭 시 → 중앙 멀티뷰에서 해당 위치 강조 및 우측 보드에 상세 정보 표시
- **최근 기록 탭**
 - 최근 48시간 내 타임아웃(미탐지)으로 제거된 생존자 이력 조회
 - 마지막 위치, 자세, 위험도, AI 상황 해석 확인 및 수동 삭제 가능

2. 중앙 메인 보드: 실시간 멀티뷰



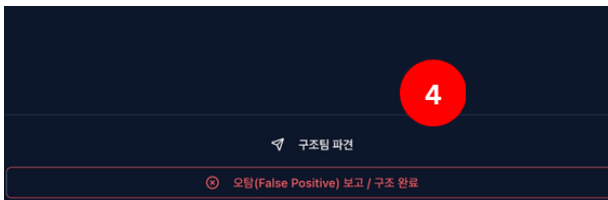
- 건물 내 다수 CCTV 영상 동시 실시간 표시
- 각 화면에 최고 위험도 점수 표시
- **객체 시각화**
 - ■ 생존자: 박스 + 자세(Pose) + 신뢰도
 - ■ 화염 / ■ 연기: 위험 요소 박스 표시
- **WiFi 센서(ESP32 CSI) 모니터링**
 - 상단 고정 CSI 신호 그래프
 - 상태 표시:
 - 탐지됨 → 최근 탐지 → 미탐지(시간 경과)
- **인터랙션**
 - 리스트 클릭 시 해당 CCTV 강조
 - 우측 보드에 상세 분석 정보 연동 표시

3. 우측 보드 : 상세 분석 정보



- **WiFi 상세:** 확대된 CSI 신호 그래프, 실시간 데이터 값, 분석 시간 확인
- **CCTV 상세:** 확대 영상, AI 신뢰도, 분석 시간, [위치/자세/상황] 해석 데이터 실시간 표시

4. 제어 및 관리 기능



- **자동 관리**
 - **자동 제거:** CCTV 화면 이탈 후 10초 이상 재탐지되지 않으면 해당 생존자를 리스트에서 자동 제거
- **수동 조치**
 - 오탐 / 구조 완료 처리
 - 오작동 또는 구조 완료 시 생존자 항목 즉시 삭제
 - 구조팀 파견 / 출동 취소
 - 출동 중 표시로 중복 파견 방지, 필요시 출동 취소 가능

[기대효과]

A. 생존 골든 타임 확보

B. 사각지대 해소

A. 생존 골든 타임 확보

Before

- 현장 내 생존자 위치 및 상태 정보가 사전에 제공되지 않음
- 구조 인력이 모든 공간을 순차적으로 수색해야 함
- 불필요한 탐색 시간 증가 및 인력 분산 발생

After

- AI 분석을 통해 생존자의 위치와 위험도를 사전에 파악
- 위험도가 높은 대상부터 구조 우선순위 부여
- 구조 인력의 이동 경로 및 투입 순서 최적화

Effect

- 불필요한 수색 시간 감소
- 초기 구조 단계에서의 의사결정 속도 향상
- 제한된 골든타임 내 구조 효율 증대

정보 부재로 모든 방을 확인



우선순위 선정 → 위급한 사람 우선



B. 사각지대 해소

Before

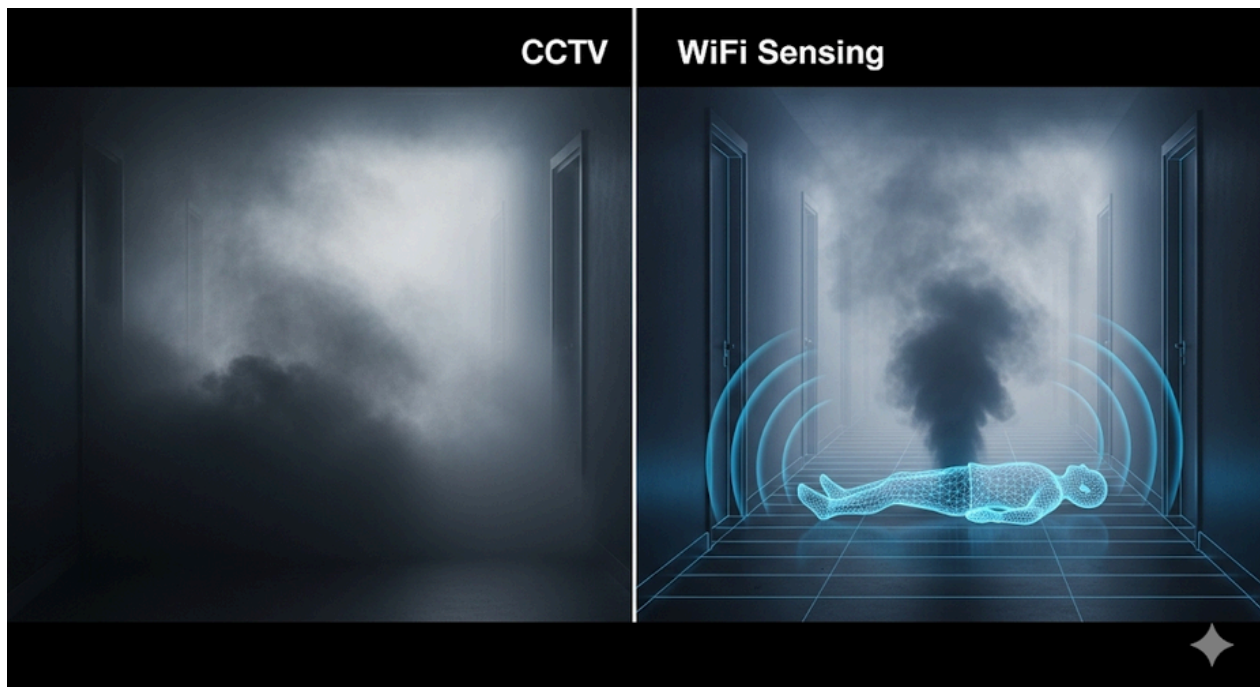
- 연기 확산 시 영상 기반 CCTV의 시야 상실
- 해당 구역에 대한 실시간 정보 획득 불가

After

- 영상 정보 확보가 어려운 경우, WiFi 신호 기반 탐지 결과 활용
- 시각 정보와 전파 기반 정보를 상호 보완적으로 사용

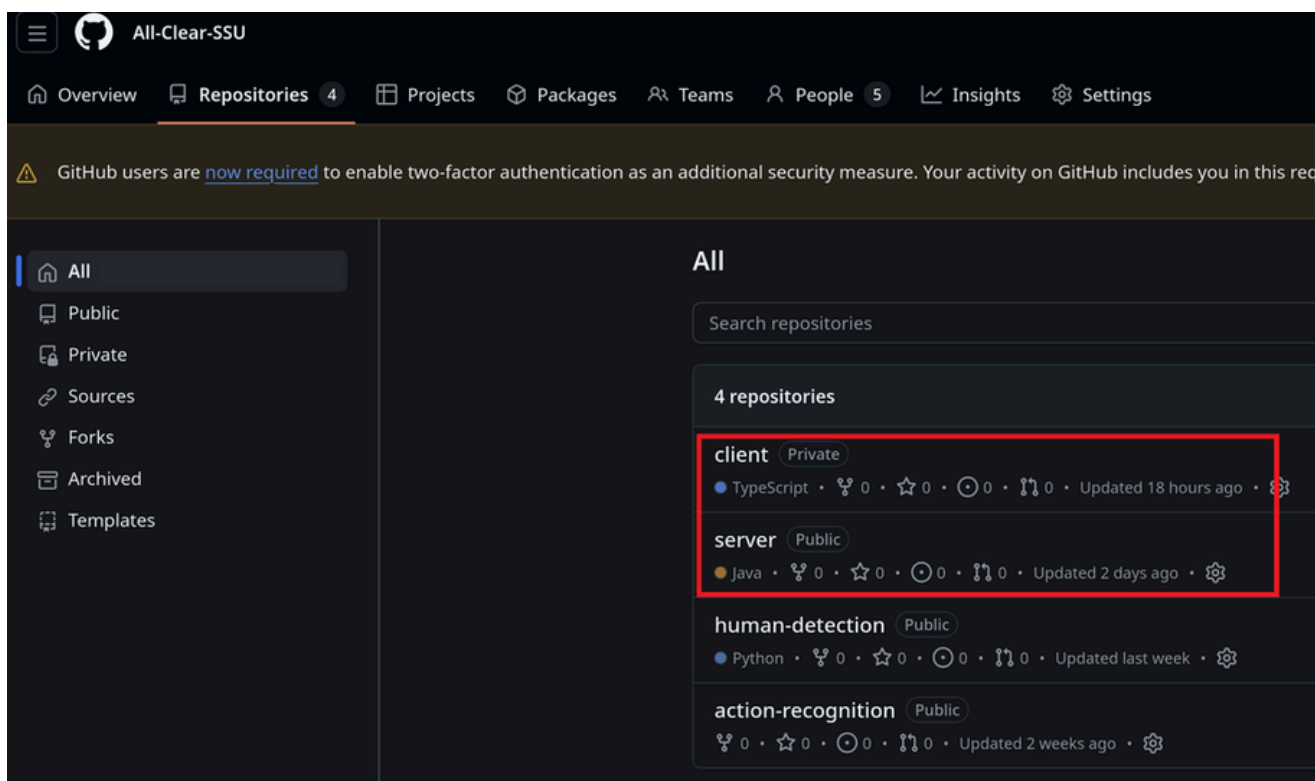
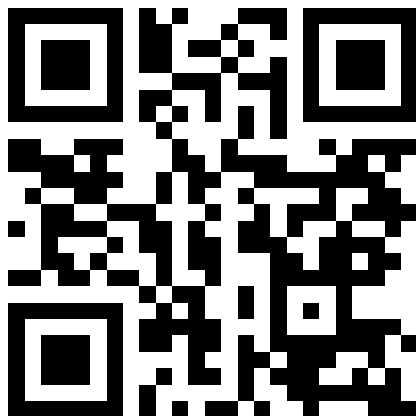
Effect

- 연기 등 시야 제한 환경에서도 인원 존재 여부 파악 가능
- 단일 센서 실패에 따른 정보 공백 최소화
- 보다 안정적인 상황 인지 유지



[Github URL]

깃허브 링크 | <https://github.com/All-Clear-SSU>



Client와 Server 상황에 맞춰서 설치